购物系统开发

项目分工汇报

组员：杜美芳 · 童昕祺 · 陈泽英 · 李炫苇 · 胡芷源（组长）

汇报内容：各模块任务分工与开发进度

目录

CONTENTS

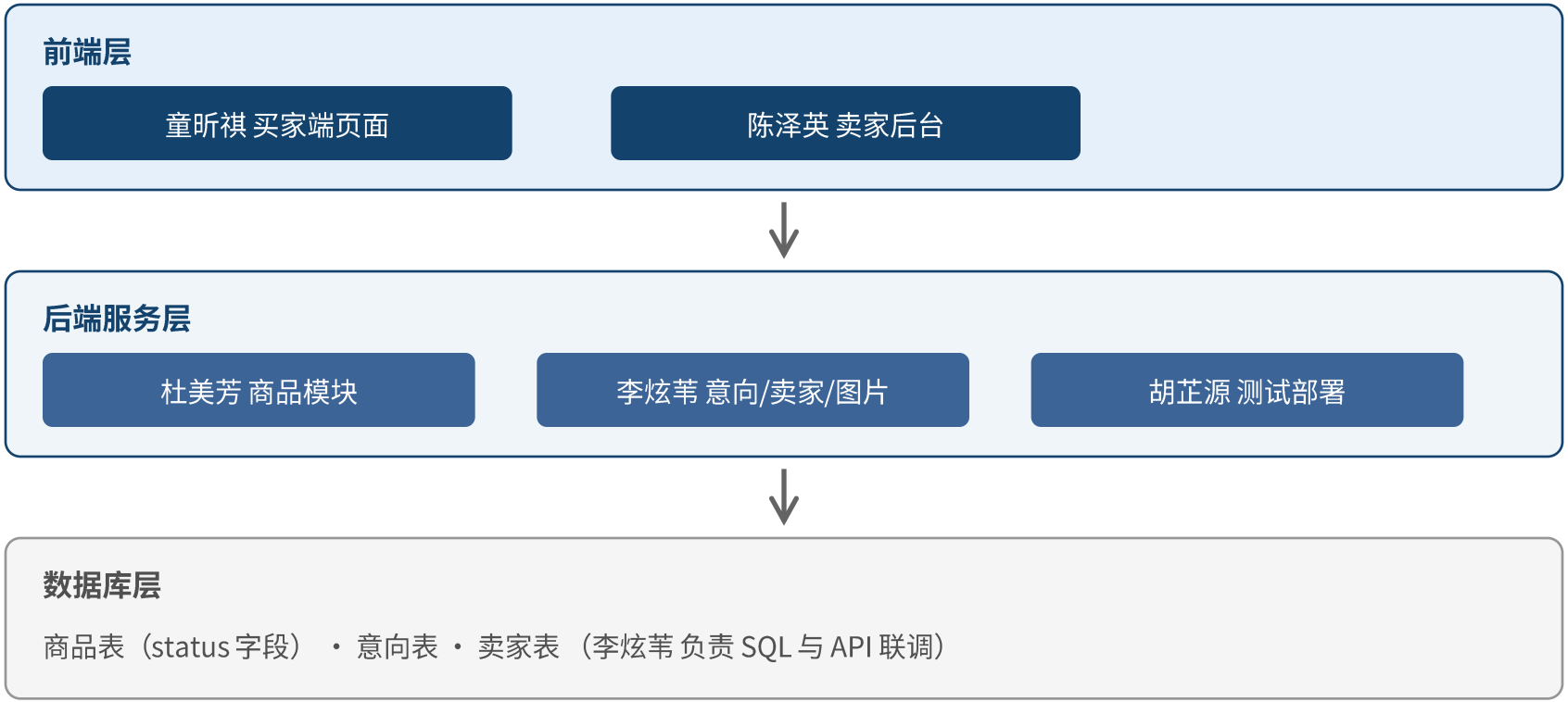
01 **项目整体架构** 前后端分工 · 数据库 · 测试部署

02 **各成员分工汇报** 杜美芳 产品经理+后端商品模块 · 童昕祺 买家端 · 陈泽英卖家后台 · 李炫苇数据库 · 胡芷源 组长+测试部署

03 **协作与接口联调** 前后端接口约定 · 联调进度

04 **下一步计划** 剩余开发任务 · 上线时间

项目整体架构


前端层
童昕祺 买家端页面
陈泽英 卖家后台
后端服务层
杜美芳 商品模块
李炫苇 意向/卖家/图片
胡芷源 测试部署
数据库层
商品表（status 字段） · 意向表 · 卖家表 （李炫苇 负责 SQL 与 API 联调）

项目整体架构

核心逻辑：商品状态机

痛点

线下交易如何防止“一物多卖”？

解决方案

设计了严谨的状态流转图。

流程

发布商品 -> 买家排队 -> 卖家触发交易（冻结） -> 线下交易 -> 成功（下架）/ 失败（自动递补下一位买家）。

项目整体架构

项目进度

目前项目整体进度约为45%

在迭代 1 的收尾阶段：我们的基线需求 MVP 已经全部跑通，包括单商品发布、意向排队、状态机流转等核心逻辑，并且 19 个后端接口全部可用。

在刚刚开启的迭代 2 中：我们面临的是“挑战需求 A”，也就是从单商品升级为多商品体系，增加库存和类别，并引入买家账号注册。目前，需求侧已经明确了 8 个功能点，产出了产品订单和冲刺订单；后端的数据模型改造正在推进；前端的页面接入也已经排上了日程。

过程管理：作为 Scrum Master，我制定了详细的任务拆解（26 个任务，共约 126 工时），并设定了燃尽图检查点。虽然目前禅道系统权限还没下发，但通过离线表格每日追踪工时，可以确保了团队进度的透明可视。

后端工作·商品模块 + 核心接口设计（基线）

杜美芳

产品经理+后端
商品模块与接口设计

后端工作

1. 商品表 status 字段设计

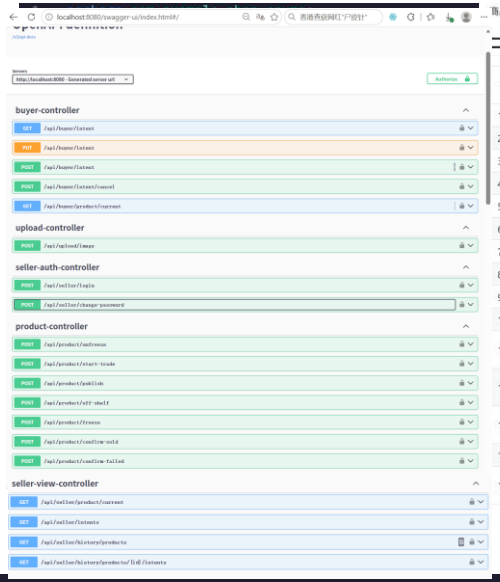
   商品状态机：发布 / 冻结 / 恢复 / 成交四种状态

2. 商品模块接口：发布 / 冻结 / 恢复 / 成交接口

   REST API 设计与状态流转逻辑

3. 15 项状态机验证全通过

   含并发递补、重新排队、交易中禁止解冻下架等异常场景



商品模块核心链路与状态机已全部验证通过，无状态错乱。以下 15 项均符合预期。

二、验证项

#	操作	接口	预期结果	实际结果
1	卖家登录	POST /api/seller/login	返回 token（code 0）	通过
2	发布商品	POST /api/product/publish	进入在售 status 0	通过
3	提交意向	POST /api/buyer/intent	排队中、返回一次性口令码、queuePosition 1	通过
4	开始交易	POST /api/product/start-trade	商品交易中冻结 status 2，队首进入交易	通过
5	交易中解冻	POST /api/product/unfreeze	拒绝，40908	通过
6	交易中下架	POST /api/product/off-shelf	拒绝，40909	通过
7	确认成交	POST /api/product/confirm-sold	商品下架 status 3；重复成交拒绝 40903	通过
8	历史商品	GET /api/seller/history/products	历史列表可见，status 3	通过
9	当前商品	GET /api/seller/product/current	无活跃商品 data null	通过
10	口令码失效	GET /api/buyer/intent?code=	商品下架后 40905	通过
11	双买家排队（先到先得）	POST /api/buyer/intent ×2	张三 position 1、李四 position 2	通过
12	交易失败递补	POST /api/product/confirm-failed (VOID)	张三作废，李四自动递补进入交易，商品保持交易中冻结，未出现中间在售态	通过
13	递补后交易中限制	POST /api/product/unfreeze	仍拒绝，40908	通过
14	递补后成交	POST /api/product/confirm-sold	李四成交，商品下架	通过
15	交易失败重新排队	POST /api/product/confirm-failed (REQUEUE)	王五转重新排队中、商品恢复在售、原口令码仍可正常查询（区别于 VOID 的口令码失效）	通过

具体成果物：
相关代码（已提交小组仓库）
后端商品模块设计文档（已写入Wiki）

```
19  public class ProductController {
35      @PostMapping("/freeze")
36      public ApiResponse<ProductDtos.SellerProductView> freeze(@Valid @RequestBody ProductDtos.ProductActionRequest req) {
37          return idempotencyService.execute(key(op: "freeze", req.requestId()),
38                  () -> ApiResponse.ok(productService.freeze(req)));
39      }
40
41      @PostMapping("/unfreeze")
42      public ApiResponse<ProductDtos.SellerProductView> unfreeze(@Valid @RequestBody ProductDtos.ProductActionRequest req) {
43          return idempotencyService.execute(key(op: "unfreeze", req.requestId()),
44                  () -> ApiResponse.ok(productService.unfreeze(req)));
45      }
```

关键代码举例：IdempotencyService.execute(key, "freeze")
幂等封装，防止卖家重复点击造成状态重复流转

产品经理工作 · 需求与过程

杜美芳

产品经理+后端
商品模块与接口
设计

挑战升级 A 需求澄清

约 80 问全部封口，冲突口径 4 处已裁决

一、多商品与队列交易（A.1-1）

#	问题	结论	标注
1	同时在售商品数量上限?	无上限	裁定
2	各商品独立队列、独立冻结?	是，互不影响	裁定
3	多商品可否同时「交易中」?	可以，多笔交易并行	裁定
4	冻结一件，其他商品可否正常浏览/提交意向?	可以，冻结只锁该商品	裁定
5	客户可否同时排多件商品的队?	可以，各队列独立	裁定
6	同一商品可否重复提交意向?	可以，不去重，按账号识别，每条独立排队	裁定
7	卖家可否跳过队首挑人?	不可以，严格先到先得；每笔交易由卖家发起时冻结	裁定
8	交易失败后队列怎么走?	队列自动前移，下一位成为新队首，但不自动进交易，须卖家发起；失败者作废/重新排队由卖家标记时确认	裁定
9	重新排队排到哪?	队尾（个别口径称未定，以林炜博明确裁定为准）	裁定
10	重新排队是否限次?	不限次	裁定
11	一次可否买多件? 有无购物车?	无购物车，一次意向=1件	裁定
12	客户可否同时购买多个商品?	可分别对多商品提交意向，无购物车式并单	裁定
13	买家可否撤销/修改/撤回意向?	不能，口令码废除后买家无此入口	裁定
14	单账号意向数量上限?	无上限	裁定
15	买家能否查询自己排第几位?	无此入口（随口令码废除），进行中意向买家端不展示，仅卖家后台可见	裁定
16	轮到买家时有无通知?	无任何通知机制	裁定
17	卖家可否一直不标记结果?	无超时机制，商品保持冻结	裁定

用户故事

迭代 2 新增 8 条故事 / 25 故事点，验收标准可验收

二、挑战需求 A 用户故事（迭代 2 · 新增）

估点口径

- 故事点采用相对估算（斐波那契 1/2/3/5/8），**1 点 ≈ 1 人 4 小时理想工时**；
- 8 条故事合计 **25 点 ≈ 100 小时**，加上联调/部署/文档约 22 小时，与冲刺订单 122 小时吻合；
- 估点方式：组内估点会逐条过，分歧大的按澄清裁定的实现复杂度复核。

故事总览

ID	故事	关联需求	故事点	优先级
A.1-1	多商品发布与独立交易	A.1-1	5	高
A.1-2	商品两级类别	A.1-2	3	中
A.1-3	库存管理与自动下架	A.1-3	3	高
A.1-4	商品浏览与搜索	A.1-4	2	中
A.1-5	多图与富媒体描述	A.1-5	3	中
A.2-1	客户注册登录（口令码废除）	A.2-1	5	高
A.2-2	客户查看历史下单记录	A.2-2	2	中
A.2-3	商家查看客户及购买历史	A.2-3	2	中
—	**合计**	—	**25**	—

https://github.com/ H15058218452-rgb/Software-Engineering-Experimental-Group/wiki/挑战升级A需求澄清问答记录

杜美芳

产品经理+后端
商品模块与接口
设计

产品订单（Product backlog）

Backlog 总表 + 与基线需求的冲突裁决

估点说明：

- 故事点：相对估算，斐波那契数列（1/2/3/5/8），1 点 ≈ 1 人 4 小时理想工时，由此估点会评定；
- Importance：100 为最高，综合需求方裁定优先级与依赖关系排定；
- 状态取值：待开发 / 进行中 / 已完成

与基线需求的冲突裁决：

基线和挑战 A 冲突的 7 处，我们逐条裁决以 A 为准，其中 4 处需求方口径本身矛盾，已记录待课程组复核之处一律以 A 为准，被取代的基线条目如下——

基线规则	迭代 2 新规则
一次只能卖一件（单活跃商品）	多商品同时在售，各自独立队列/交易
免注册 + 口令码认领意向	必须注册登录，口令码废除
买家可凭口令码修改/撤销意向	买家无修改、无撤销入口
提交意向必填姓名+电话	意向信息取自账号（用户名+电话），无姓名字段
历史只读（卖家侧）	历史只读扩展到买家侧（我的订单）
成交即下架（单件语境）	成交减库存，减到 0 才下架
交易失败自动递补	队列自动前移，但进交易须卖家发起；失败者作废/重排由卖家确认

Backlog（迭代 2 · 挑战需求 A）

ID	Name	故事点	Importance	How to demo	Notes	状态
A.1-1	多商品发布与独立交易	5	100	同时发布 3 件商品各自收意向；两件同时进入交易互不影响；失败标重新排队后排到队尾	移除单活跃约束；队列/冻结/交易按商品独立；先到先得不可挑人	待开发
A.2-1	客户注册登录（口令码废除）	5	95	游客浏览→点购买引导注册→登录后提交意向并挂账号；界面上不存在口令码	用户名唯一+密码≥8位+电话必填不唯一+默认交易地点选填；口令码整体废除、旧意向不迁移	待开发
A.1-3	库存管理与自动下架	3	90	发布库存 2→成交 1 次减 1→减到 0 自动下架消失→补货后手动重新上架	仅成交扣减；手动加减不可负；售罄队列意向失效不复活；不超卖	待开发
A.1-2	商品两级类别	3	70	后台建"服装→女装"→发布商品挂二级→买家按"女装"筛选	固定两级、卖家自维护、发布必填；有商品的类别不可删	待开发
A.1-5	多图与富媒体描述	3	60	上传 9 张图（首张为主图）→列表显示主图→详情看全部；描述内嵌图片视频	JPG/PNG≤5MB、选填有占位图；富文本需 XSS 白名单	待开发
A.1-4	商品浏览与搜索	2	65	列表分页浏览→按类别筛选→关键词搜名称/描述→点开详情	仅在售；结果含名称/价格/主图/类别；分页排序按组内拟定口径	待开发
A.2-2	客户查看历史下单记录	2	75	登录→我的订单→看倒序记录（名称/下单时间/结果/交易时间）→翻页	提交即下单；未出结果标「待交易」（冲突项已标注）；只读、快照保留	待开发
A.2-3	商家查看客户及购买历史	2	65	后台客户列表→看用户名/电话/地点/注册时间→点开看购买历史；按商品看意向购买人	注册即可见；不可导出/拉黑；仅限交易撮合用途	待开发
—	**合计**	**25**	—	—	25 点 ≈ 100h 理想工时；含联调/部署/文档约 122h，见冲刺订单	—

产品经理工作·线框图 +成果展示

杜美芳

产品经理+后端
商品模块与接口
设计

线框图：迭代 2 的 8 个关键页面

买家端 5 页 + 卖家端 3 页

过程/成果文档（截止10.21）

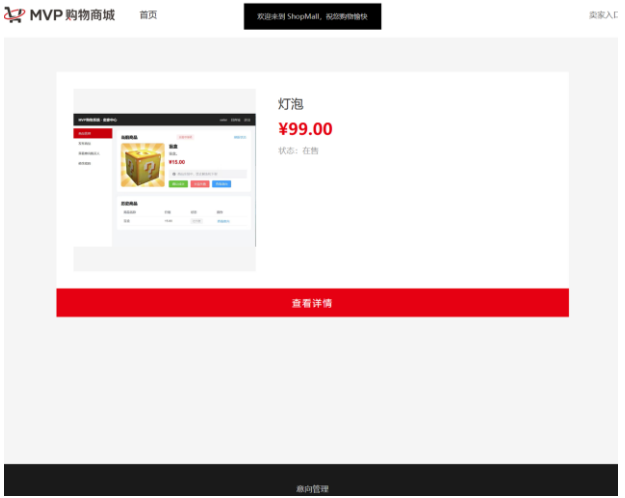
- 挑战升级A需求澄清问答记录
- 用户故事（迭代2·挑战需求A）
- 产品订单Product Backlog
- 冲刺订单Sprint Backlog（初稿已编制，待待全组迭代计划会评审确认后确认）



童昕祺

买家端

商品浏览首页



页面结构：顶部欢迎条 → 头部导航（Logo / 首页 / 卖家入口）→ 商品卡片 → 底部意向管理入口

loadProduct函数是首页的数据加载核心。打开页面自动请求商品接口，用 try-catch-finally 结构，成功拿到商品信息；出错时捕获异常给出提示

```
async function loadProduct() {
  loading.value = true
  error.value = ''
  try {
    product.value = await getCurrentProduct()
  } catch (e) {
    error.value = e.message || '加载商品失败，请重试'
  } finally {
    loading.value = false
  }
}
```

童昕祺
买家端

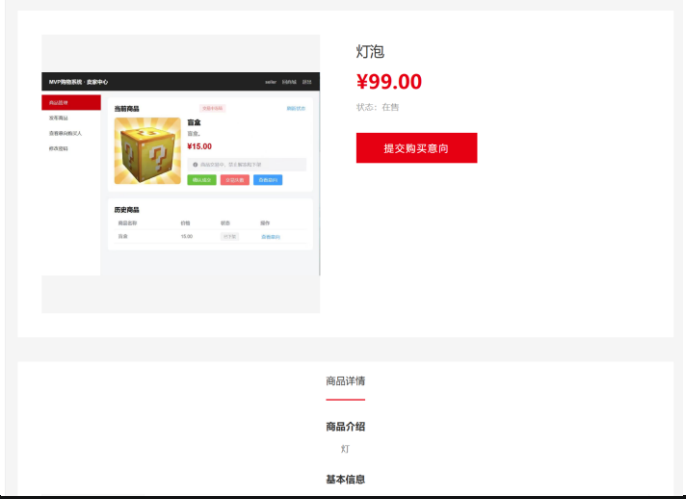
商品详情页

页面结构：顶部导航（Logo / 首页 / 卖家入口） → 面包屑导航 → 商品信息卡片（左大图 + 右侧名称/价格/动态状态） → 商品详情区（Tab页签、图文介绍、参数表格） → 底部意向管理入口。

核心逻辑与状态控制：
状态机 UI 映射：根据后端返回的 product.status 映射中文状态字典；根据 canSubmitIntent 字段动态控制“提交购买意向”按钮的置灰禁用状态与提示文案。

安全加载与异常兜底：loadProduct 函数配合 loading 和 error 响应式变量，实现加载中提示、接口报错捕获及空数据状态（暂无在售商品）的友好展示。

组件化拆分：将复杂的意向提交与口令码展示逻辑，抽离封装为独立的 SubmitIntentModal 弹窗组件，保持页面代码整洁。




```
<div v-if="!product.canSubmitIntent" class="tips">
  商品交易中，暂时无法提交购买意向
</div>

<button
  class="buy-btn"
  :disabled="!product.canSubmitIntent || submitting"
  @click="goSubmit"
>
  {{ product.canSubmitIntent ? '提交购买意向' : '商品交易中' }}
</button>
</div>
```

童昕祺
买家端

购买意向提交弹窗

交互设计：采用 Modal 弹窗形式，通过 step 变量在同一组件内切换“表单填写”和“成功展示”两个视图，避免页面跳转。

核心逻辑与异常处理：
前端校验：使用 trim 去除空格，利用正则表达式校验 11 位手机号格式，姓名长度限制为 20 字符。

防抖与异常兜底：submitting 状态防止用户连点；在 catch 中针对后端特定的错误码（如 40904 重复提交）进行 UI 提示。






```
async function handleSubmit() {
  if (submitting.value) return
  if (!validateAll()) return

  submitting.value = true
  try {
    const res = await submitIntent({
      productId: props.product?.id,
      name: form.value.name.trim(),
      phone: form.value.phone.trim()
    })
    intentCode.value = res.code
    submitTime.value = new Date().toISOString()
    step.value = 'success'
    emit('success')
```

```
    // 成功视图出现后，立即弹出 Toast，2 秒后消失
    showToast.value = true
    if (toastTimer) clearTimeout(toastTimer)
    toastTimer = setTimeout(() => {
      showToast.value = false
      toastTimer = null
    }, 2000)
  } catch (e) {
    if (e.code === 40904) {
      errors.value.phone = '您已提交过该商品的购买意向，请勿重复提交'
    } else {
      errors.value.name = e.message || '提交失败，请重试'
    }
  } finally {
    submitting.value = false
  }
}
```

童昕祺

买家端

意向管理页

页面结构：顶部提示语告知用户输入口令码的目的→核心输入框→红色“查询意向”主按钮→底部提供返回首页的链接。

用户输入口令码查到数据后，口令码存入localStorage，跳转到意向详情页时，后续所有的修改和撤销接口，都能自动带上口令码，不用用户再输一次；前端针对后端的40905失效错误码做了专门的拦截，如果用户输入了过期的口令码，页面会给出明确的提示。




```
async function handleQuery() {
  if (loading.value) return

  const v = validateCode()
  if (v) {
    error.value = v
    return
  }

  loading.value = true
  error.value = ''
  try {
     const res = await queryIntent(code.value.trim())
     localStorage.setItem(STORAGE_KEYS.INTENT_CODE, code.value.trim())
     router.push('/buyer/intent/detail')
  } catch (e) {
    if (e.code === 40905) {
      error.value = '口令码已失效，无对应意向信息'
    } else {
      error.value = e.message || '查询失败，请重试'
    }
  } finally {
    loading.value = false
  }
}
```

前端买家端

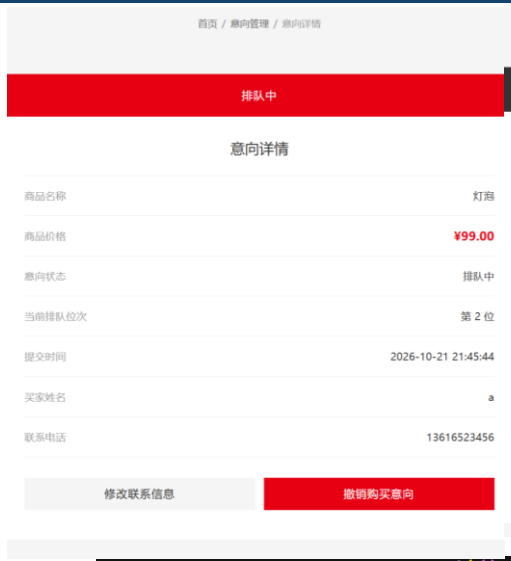
童昕祺

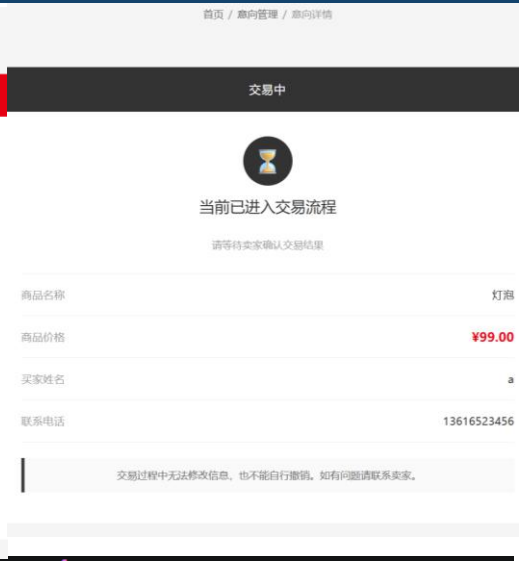
买家端

意向详情页

本页面采用“状态机驱动”的设计理念。前端根据后端返回的意向状态，自动切换不同的页面视图。当状态为“排队中”时，展示当前排队位次，并开放修改与撤销操作；当进入“交易中”时，系统会强制禁用所有操作按钮，防止数据冲突；若“已失效”，仅引导用户返回首页。

为了保证操作安全，修改联系信息与撤销意向均采用了弹窗二次确认机制，其中修改信息不会影响用户原有的排队顺序。同时，页面通过计算属性联动顶部状态条的颜色，实现视觉与状态的统一。此外，在数据加载环节，系统不仅拉取意向详情，还会异步获取最新的商品价格与名称进行合并展示，确保用户看到的信息实时准确。






```
const statusText = computed(() => {
  if (!intent.value) return ''
  return INTENT_STATUS_TEXT[intent.value.intentStatus] || '未知'
})

const canOperate = computed(() => {
  const s = intent.value?.intentStatus
  return s === INTENT_STATUS.QUEUING || s === INTENT_STATUS.REQUEUED
})

const isTrading = computed(() => {
  return intent.value?.intentStatus === INTENT_STATUS.TRADING
})
```

陈泽英

卖家后台

整体页面开发与登录拦截

1. 完成情况

完成了卖家后台（登录、发布商品、商品管理、意向管理、修改密码）全部 5 个页面的开发和接口对接。

2. 登陆安全

在发请求时自动带上登录凭证（Token），如果没登录，直接拦截在后台门外，跳回登录页。

3. 统一请求

封装了网络请求，统一处理各种报错提示。




```
request.interceptors.request.use(config => {
  config.headers['X-Request-Id'] = genRequestId()

  const token = localStorage.getItem(STORAGE_KEYS.SELLER_TOKEN)
  if (token) {
    config.headers['Authorization'] = `Bearer ${token}`
  }
```

陈泽英

卖家后台

商品状态切换与按钮控制

1. 商品状态

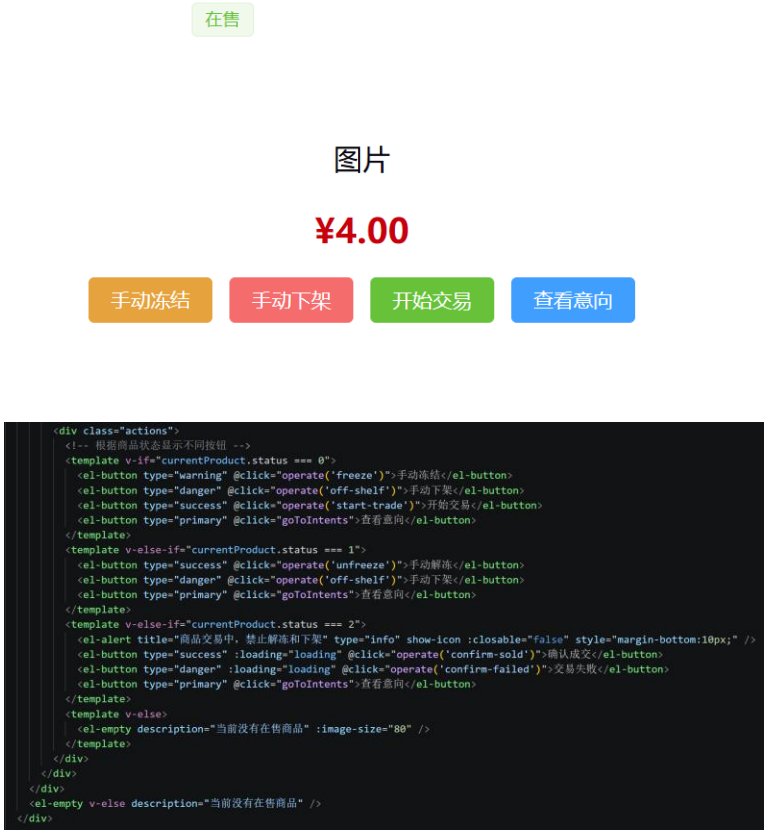
实现了“在售、冻结、交易中、已下架”四种状态的页面展示。

2. 按钮控制

前端严格按照后端返回的数据来显示对应的操作按钮

3. 防重复提交

点击"立即发布"后，按钮会自动转圈（Loading），防止用户点两次发出重复请求。




```
<div class="actions">
  <!-- 根据商品状态显示不同按钮 -->
  <template v-if="currentProduct.status === 0">
    <el-button type="warning" @click="operate('freeze')">手动冻结</el-button>
    <el-button type="danger" @click="operate('off-shelf')">手动下架</el-button>
    <el-button type="success" @click="operate('start-trade')">开始交易</el-button>
    <el-button type="primary" @click="goToIntents">查看意向</el-button>
  </template>
  <template v-else-if="currentProduct.status === 1">
    <el-button type="success" @click="operate('unfreeze')">手动解冻</el-button>
    <el-button type="danger" @click="operate('off-shelf')">手动下架</el-button>
    <el-button type="primary" @click="goToIntents">查看意向</el-button>
  </template>
  <template v-else-if="currentProduct.status === 2">
    <el-alert title="商品交易中，禁止解冻和下架" type="info" show-icon :closable="false" style="margin-bottom:10px;" />
    <el-button type="success" :loading="loading" @click="operate('confirm-sold')">确认成交</el-button>
    <el-button type="danger" :loading="loading" @click="operate('confirm-failed')">交易失败</el-button>
    <el-button type="primary" @click="goToIntents">查看意向</el-button>
  </template>
  <template v-else>
    <el-empty description="当前没有在售商品" :image-size="80" />
  </template>
</div>
</div>
</div>
<el-empty v-else description="当前没有在售商品" />
</div>
```

陈泽英

卖家后台

交易确认与异常处理

1. 传参排错

解决点击“确认成交”没反应的问题，通过查询意向列表，拿到了正确的买家意向 ID（intentId）传给后端。

2. 弹窗选择

按照需求文档，交易失败时前端弹出选择框，让卖家选择是“作废”还是“重新排队”。

3. 异常捕获机制

所有关键操作都加了 try...catch 异常拦截，如果后端报错，页面会弹出红色提示，不会白屏死机。




```
// 2. 如果是交易失败，必须要弹窗让卖家选择处置方式
if (action === 'confirm-failed') {
  try {
    await ElMessageBox.confirm('请选择交易失败后的处理方式', '交易失败', {
      distinguishCancelAndClose: true,
      confirmButtonText: '作废 (VOID)',
      cancelButtonText: '重新排队 (REQUEUE)',
      type: 'warning'
    })
    payload.disposition = 'VOID' // 点了确定，作废
  } catch (error) {
    if (error === 'cancel') {
      payload.disposition = 'REQUEUE' // 点了取消，重新排队
    } else {
      loading.value = false
      return // 点了右上角关闭，直接中止
    }
  }
}
```

陈泽英

卖家后台

前端表单限制与用户体验

1.发布商品限制

在前端限制只能上传 JPG/PNG 格式的图片，且不能超过 5MB；对价格做正则校验，限制必须大于 0 且最多两位小数。

2. 修改密码限制

严格校验两次新密码输入是否一致，且新密码长度不得少于 8 位。

3. 错误提示反馈

用户只要填错信息，页面上立刻弹出红色错误提示（如“价格必须大于0”），提升用户体验。




```
const onSubmit = async () => {
  // 1. 必填校验
  if (!form.name || !form.price) {
    flash('请完整填写名称和价格', true)
    return
  }

  // 2. 价格格式校验（>0，最多两位小数）
  const priceStr = String(form.price)
  if (!/^\d+(\.\d{1,2})?$/.test(priceStr) || Number(priceStr) <= 0) {
    flash('价格必须大于0，且最多两位小数', true)
    return
  }
```

陈泽英

卖家后台

前后端联调与数据展示

1. 解决传参报错

在联调时排查并解决了一个前端报错（传参数传错），成功让页面拿到了真实的商品 ID。

2. 数据对齐

把后端传来的复杂数据，正确解析并渲染到了前端表格里。

3. 时间格式美化

后端传过来的时间格式不是整洁的时间（像 2026-10-21T15:07:03），前端把它截取并变成了整洁的“年-月-日 时:分:秒”。




```
// 2. 格式化时间函数：把 2026-10-21T15:07:03+08:00 变成 2026-10-21 15:07:03
const formatTime = (timeStr) => {
  if (!timeStr) return '-'
  // 截取前 19 个字符，替换中间的 'T' 为空格
  return timeStr.substring(0, 19).replace('T', ' ')
}
```

李炫苇

数据库

讲述大纲

1. 数据库表设计
   数据库共 3 张表：`seller`（卖家）、`product`（商品）、`intent`（购买意向），MySQL 8.x，InnoDB + utf8mb4。

2. 图片上传接口
`POST /api/upload/image`，multipart/form-data，字段名`file`，需卖家登录（Bearer Token）。
- 校验： 仅允许 jpg / jpeg / png / webp，单张 ≤ 5MB，非法类型/超限返回业务错误码；
- 存储： 文件重命名为 UUID（防重名、防路径注入），保存到可配置目录（`app.upload-dir`），通过静态资源映射`/uploads/**`对外提供访问；
- 返回： `{"code":0, "message":"ok", "data":{"url":"/uploads/xxx.png"}}`，前端拼后端域名即可展示；
- 排障实战： 联调中发现"上传报保存失败 + 图片 404"两个 bug——定位到静态资源映射路径缺结尾`/`导致全部 404，以及安全软件拦截后端写项目目录；修复后上传→访问→发布→买家端展示全链路实测通过。

```
单活跃商品约束（product 表） — 摘自 shop_schema.sql
active_flag TINYINT GENERATED ALWAYS AS

  (CASE WHEN status IN (0,1,2) THEN 1 ELSE NULL END)
STORED

  COMMENT '未下架标记，配合唯一索引实现单活跃商品约束',

卖家表（seller 表） — 摘自 shop_schema.sql
  UNIQUE KEY uk_product_active (active_flag)
CREATE TABLE IF NOT EXISTS seller (

  id          BIGINT      NOT NULL AUTO_INCREMENT,

  username    VARCHAR(50)  NOT NULL COMMENT '登录用户名（固定单账号）',

  password_hash VARCHAR(100) NOT NULL COMMENT 'BCrypt 加密密码',

  created_at  DATETIME(3)  NOT NULL DEFAULT
CURRENT_TIMESTAMP(3),
```

李炫苇

数据库

讲述大纲

意向表（intent表）（口令码哈希 + 队列索引） — 摘自 shop_schema.sql

```
CREATE TABLE IF NOT EXISTS intent (
    id          BIGINT      NOT NULL AUTO_INCREMENT,
    product_id  BIGINT      NOT NULL COMMENT '所属商品ID',
    code_hash   CHAR(64)    NOT NULL COMMENT '口令码 SHA-256 哈希（不存明文）',
    name        VARCHAR(50) NOT NULL,
    phone       VARCHAR(20) NOT NULL,
    status      TINYINT     NOT NULL DEFAULT 0 COMMENT '0排队中 1交易中 2成功 3作废 4撤销 5重排 6最终失败',
    queued_at   DATETIME(3) NOT NULL COMMENT '排队时间，先到先得排序依据',
    fail_count  INT         NOT NULL DEFAULT 0,
    PRIMARY KEY (id),
    UNIQUE KEY uk_intent_code (code_hash),
    KEY idx_intent_queue (product_id, status, queued_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

后端 + 数据库

讲述大纲

图片上传接口 —— 摘自 UploadService.java

```
public CommonDtos.UploadResponse upload(MultipartFile file) {
    if (file == null || file.isEmpty())
        throw new BizException(ErrorCode.PARAM_ERROR.getCode(), "请选择要上传的图片");
    if (file.getSize() > MAX_SIZE)  // 5MB
        throw new BizException(ErrorCode.PARAM_ERROR.getCode(), "图片大小不能超过 5MB");
    String ext = original.substring(original.lastIndexOf('.') + 1).toLowerCase();
    if (!ALLOWED.contains(ext))     // jpg / jpeg / png
        throw new BizException(ErrorCode.PARAM_ERROR.getCode(), "仅支持 JPG/PNG 格式图片");
    Files.createDirectories(uploadDir);
    String filename = UUID.randomUUID().toString().replace("-", "") + "." + ext;
    try (InputStream in = file.getInputStream()) {
        Files.copy(in, uploadDir.resolve(filename), StandardCopyOption.REPLACE_EXISTING);
    }
    return new CommonDtos.UploadResponse("/uploads/" + filename);  // 返回可访问 URL
}
```

配套静态资源映射 WebConfig.java：

```
String location = Paths.get(uploadDir).toAbsolutePath().normalize().toUri().toString();
if (!location.endsWith("/")) location += "/";  // 关键：缺 "/" 会导致 /uploads/** 全部 404
registry.addResourceHandler("/uploads/**").addResourceLocations(location);
```

李炫苇

数据库

李炫苇

数据库

讲述大纲

3. SQL 与接口实现

交付独立可执行的数据库脚本与配套接口。

- SQL 脚本：`shop_schema.sql` ——三表 DDL + 默认卖家种子数据，幂等（`IF NOT EXISTS` +`INSERT IGNORE`），可手动执行也可由应用启动自动建表；
- 意向接口：提交（发口令码、算位次）、查询（口令码查状态/位次/商品名/价格）、修改、撤销，状态机驱动，作废自动递补下一位；
- 卖家接口：登录发 JWT、发布商品、开始交易、确认成交/失败、历史查询；
- 统一响应：`{code, message, data}`，业务错误也返回 HTTP 200，前端只看 code；错误码如 40904 重复提交、40908/40909 冻结保护、40905 口令码失效。

SQL 与接口实现 — 意向提交（悲观锁 + 查重 + 算位次） — 摘自 BuyerService.java

```
UPDATE 行锁

    if (p == null) throw new BizException(ErrorCode.NOT_FOUND.getCode(), "暂无在售商品");

    if (p.getStatus() != ProductStatus.ON_SALE.getCode())

        throw new BizException(ErrorCode.PRODUCT_FROZEN.getCode(), "商品已冻结/交易中，无法提交意向");

    if (intentMapper.countActiveByPhone(p.getId(), req.phone().trim()) > 0)

        throw new BizException(ErrorCode.DUPLICATE_INTENT);  // 同手机号防重复提交

    String code = generateCode();                    // 12 位口令码，明文只返回这一次

    Intent it = new Intent();

    it.setCodeHash(sha256(code));                    // 库中只存 SHA-256 哈希
it.setQueuedAt(LocalDateTime.now().truncatedTo(ChronoUnit.MILLIS));

    intentMapper.insert(it);
```

李炫苇

数据库

讲述大纲

4. API 联调

编写 PowerShell 冒烟测试脚本（32 项断言），与 A、B、C 三部分真实联调。
覆盖完整业务链：登录 → 发布商品 → 3 买家排队（位次1/2/3）→ 口令码查询 → 重复提交拦截 → 开始交易 → 交易中禁止解冻/下架 → 失败作废自动递补 → 成交下架 → 历史可查 → 口令码失效。
实测环境 JDK + MySQL 8.4.7，32/32 全部通过。

联调中推动解决了 3 个跨端问题：图片裂图（后端 2 个 bug）、意向详情缺商品名称/价格字段（补`productName` /`productPrice` /`createdAt`）、前端`status`映射不全（改用`statusText`）。

```
# Step 6: 3 buyers submit intents, verify FIFO positions 1/2/3

Check "Buyer1 position = 1" ($b1.data.intent.queuePosition -eq 1)
```

— 摘自 api-smoke-test.ps1

```
Check "Buyer2 position = 2" ($b2.data.intent.queuePosition -eq 2)

# Step 7: duplicate phone rejected

Check "Duplicate submit -> 40904" ($dup.code -eq 40904)

# Step 9: trading freeze protection

Check "Freeze during trading -> 40908" ($frz.code -eq 40908)

Check "Off-shelf during trading -> 40909" ($off.code -eq 40909)

# Step 10: confirm-failed -> VOID, next buyer auto-promoted, product stays 2
```

测试 + 部署

胡芷源

组长+
测试部署

测试方法：黑盒测试（基于业务流）+接口测试（Postman 自动化脚本）。
覆盖维度：核心业务流程（Happy Path）+异常边界（Unhappy Path）。
成果产出：在 Wiki 编写了 18 条完整测试用例（涵盖重复提交、冻结期购买、越权操作等）。

缺陷追踪与 Bug 修复：

案例：在联调时，测试出“卖家登录报错 token is null”以及“商品图片渲染为纯文本”。
处理流程：在 GitHub 提交 Issue 指派给对应开发 -> 协助排查原因（前端标签语法错误） -> 修复并复测通过 -> 关闭 Issue。

部署方案与 Docker 容器化：

本地环境：JDK 17 + MySQL 8.0 + Node.js。
容器化配置：
写了完整的 Dockerfile 与 docker-compose.yml，实现前后端与数据库的一键构建。
自动化脚本：后端提供的 api-smoke-test.ps1 实现了 32 项接口冒烟测试（附上全绿截图）。

迭代验收标准：

核心闭环跑通（发布->购买->交易->完成）。
异常状态成功拦截（商品冻结期间无法重复提交）。
前端无严重崩溃白屏，数据库数据正确落库。

胡芷源

组长+
测试部署

项目难点与复盘：

难点 1（环境）：本地 Java 27 与代码要求的 Java 17 冲突，通过配置局部 JAVA_HOME 解决。

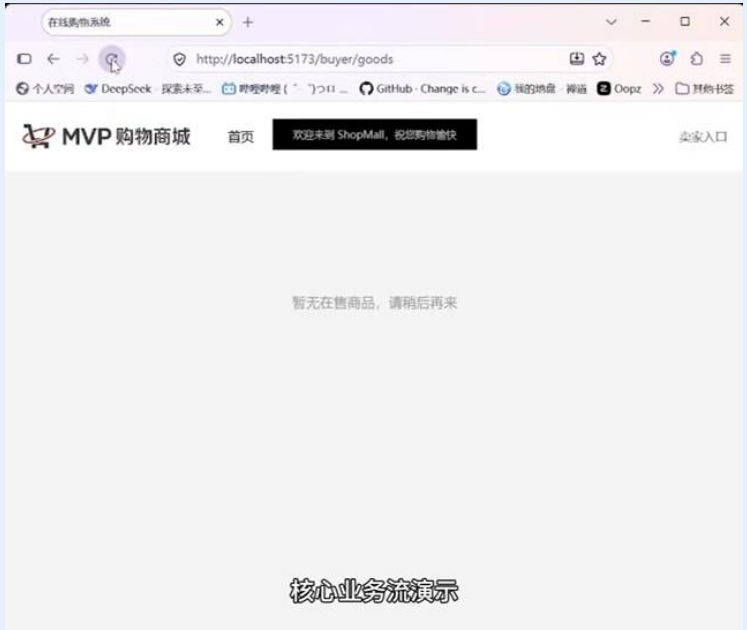
```
PS D:\Workspace\software-project\Software-Engineering-Experimental-Group\backend> $env:JAVA_HOME="D:\jdk17\jdk-17.0.16_windows-x64_bin\jdk-17.0.16"
```

难点 2（协同）：Git 分支合并冲突，漏传 package.json 导致前端跑不起来。通过强制拉取和补齐文件解决。

难点 3（网络）：Docker Hub 国内网络受限，采用提供完整 Dockerfile 及 docker-compose 的方式供本地构

docker-compose.yml	Add files via upload	2 days ago
Dockerfile	Add files via upload	

系统界面与核心功能演示



完整录屏已上传至百度网盘及 Wiki

协作与接口联调

接口约定

买家端 ←→ 商品接口 + 意向接口

卖家后台 ←→ 商品 + 卖家/意向接口

测试 ←→ 所有接口（Postman 集合）

联调进度

- 商品列表 / 详情接口：已联调
- 意向提交接口：已联调
- 商品冻结/恢复接口：已联调
- 卖家登录/改密码：已联调

协作方式：统一接口文档（字段名 / 状态码 / 返回格式），
前后端并行开发，接口 Mock 先行，后续可以用 Postman 回归测试。

感谢聆听

购物系统开发 第六小组